B. Rakhsh's Revival

time limit per test 1 second
memory limit per test 256 megabytes
input standard input
output standard output

Rating: 1000 | Tags: data structures, greedy, implementation, two pointers |
https://codeforces.com/problemset/problem/2034/B

Rostam's loyal horse, Rakhsh, has seen better days. Once powerful and fast, Rakhsh has grown weaker over time, struggling to even move. Rostam worries that if too many parts of Rakhsh's body lose strength at once, Rakhsh might stop entirely. To keep his companion going, Rostam decides to strengthen Rakhsh, bit by bit, so no part of his body is too frail for too long.



Imagine Rakhsh's body as a line of spots represented by a binary string s of length n, where each 0 means a weak spot and each 1 means a strong one. Rostam's goal is to make sure that no interval of m consecutive spots is entirely weak (all 0s).

Luckily, Rostam has a special ability called Timar, inherited from his mother Rudabeh at birth. With Timar, he can select any segment of length k and instantly strengthen all of it (changing every character in that segment to 1). The challenge is to figure out the minimum number of times Rostam needs to use Timar to keep Rakhsh moving, ensuring there are no consecutive entirely weak spots of length m.

Input

The first line contains an integer t ($1 \le t \le 10^4$), the number of test cases.

The first line of each test case contains three numbers n, m, k ($1 \le m, k \le n \le 2 \cdot 10^5$). The second line of each test case contains a binary string s of n characters $s_1 s_2 \ldots s_n$. ($s_i \in \{0,1\}$ for $1 \le i \le n$).

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output the minimum number of times Rostam needs to use Timar to keep Rakhsh moving, ensuring there are no consecutive entirely weak spots of length m.

Example

Input
3 5 1 1 10101 5 2 1 10101 6 3 2 000000

Output
2 0 1

Note

In the first test case, we should apply an operation on each `0`.

In the second test case, s is already ok.

In the third test case, we can perform an operation on interval [3,4] to get `001100`.